

Data Mining

HW 4

1) Multilayer Neural Network

(a) Encoding the categorical data and ordinal data in general:

To input the categorical and ordinal data into a neural network, we need to transform non-numeric attributes into a suitable numeric format.

General Approach for encoding:

- Categorical data (e.g., department, status) is typically encoded using one-hot encoding, where each category is represented as a binary vector. It converts each category into a binary column (0 or 1) for each unique label in the dataset.

Example:

Department (one-hot encoding):

- ❖ Sales $\Rightarrow$ [1, 0, 0, 0]
- ❖ Systems $\Rightarrow$ [0, 1, 0, 0]
- ❖ Marketing $\Rightarrow$ [0, 0, 1, 0]
- ❖ Secretary $\Rightarrow$ [0, 0, 0, 1]

- Ordinal data (e.g., age ranges, salary ranges) is best represented with ordinal encoding if there is a clear order, or one-hot encoding if the levels don't have a clear hierarchy. It assigns each category a numeric value based on its rank or order.

Example:

Age (ordinal):

- ❖ Age: {'21-25': 0, '26-30': 1, '31-35': 2, '36-40': 3, '41-45': 4, '46-50': 5}

For specific types like Department, Age, Salary, Status:

Department:

- ❖ Sales $\Rightarrow$ [1, 0, 0, 0]
- ❖ Systems $\Rightarrow$ [0, 1, 0, 0]
- ❖ Marketing $\Rightarrow$ [0, 0, 1, 0]
- ❖ Secretary $\Rightarrow$ [0, 0, 0, 1]

Age:

Since age is ordinal with ranges, we can assign numeric labels to each age range, such as:

- ❖ Age: {'21-25': 0, '26-30': 1, '31-35': 2, '36-40': 3, '41-45': 4, '46-50': 5}

Salary:

Similarly, we can assign numeric labels to each salary range:

- ❖ Salary: {'26K-30K': 0, '31K-35K': 1, '36K-40K': 2, '41K-45K': 3, '46K-50K': 4, '66K-70K': 5}

Class Label: Status:

This is the output variable, so we can use binary encoding for status:

- ❖ junior $\Rightarrow$ 0
- ❖ senior $\Rightarrow$ 1

(b) I want to design 1 hidden layer and I want to design 4 input features.

Number of hidden layer : 1

Number of nodes in hidden layer : 4

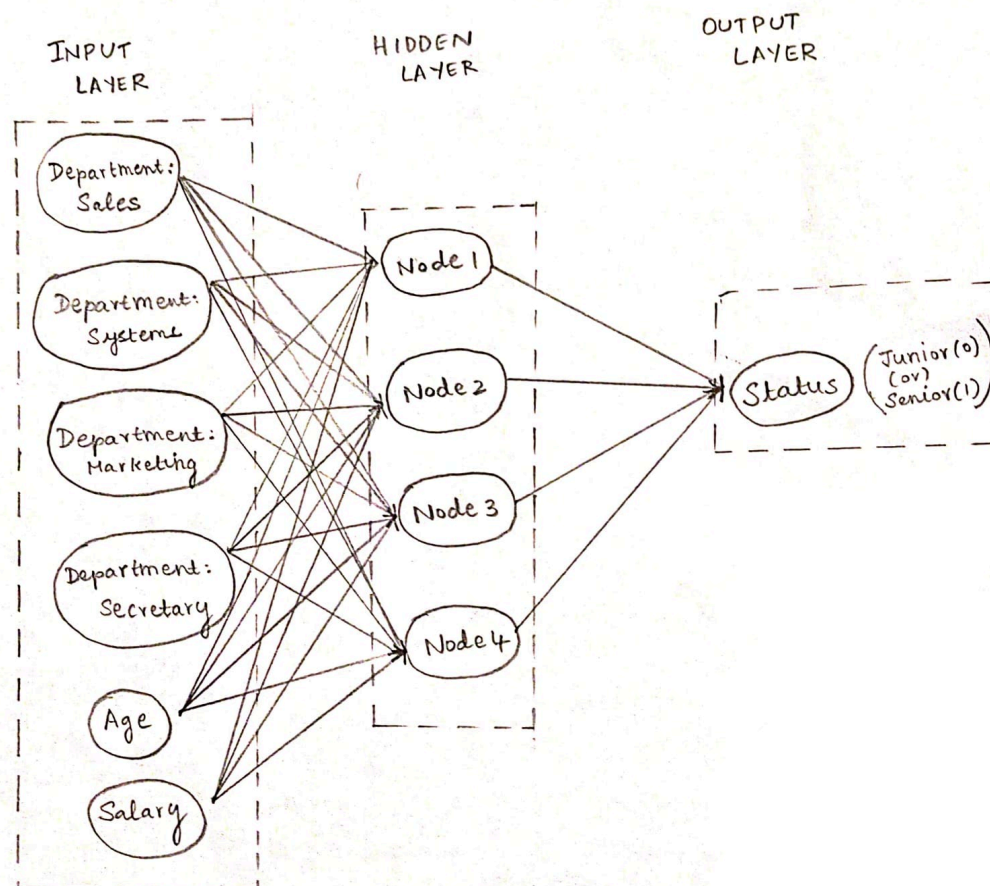
Number of nodes in input layer : 6 (Department: 4 binary feature + Age: 1 binary feature + Salary: 1 binary feature)

Number of nodes in output layer : 1 node for binary classification (status - junior(0) or senior(1))

For input layer:

- Department (Categorical - Nominal):
We have four unique values here: Sales, Systems, Marketing, and Secretary. With One-Hot Encoding, this will create 4 binary features (one for each department).
- Age (Ordinal):
There are six possible age ranges (21-25, 26-30, 31-35, 36-40, 41-45, 46-50). Using Ordinal Encoding, we can represent this as a single numerical feature, with values from 1 to 6 based on the order of the age ranges.
- Salary (Ordinal):
There are six possible salary ranges (26K-30K, 31K-35K, 36K-40K, 41K-45K, 46K-50K, 66K-70K). Using Ordinal Encoding, we can represent this as another single numerical feature, with values from 1 to 6 based on the order of the salary ranges.

(c) ANN Model:



(d) Code:

```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import OneHotEncoder
import numpy as np

# Encoding categorical and ordinal data and Defining the data
data = [
    ['Sales', 'senior', '31-35', '46K-50K', 30],
    ['Sales', 'junior', '26-30', '26K-30K', 40],
    ['Sales', 'junior', '31-35', '31K-35K', 40],
    ['Systems', 'junior', '21-25', '46K-50K', 20],
    ['Systems', 'senior', '31-35', '66K-70K', 5],
    ['Systems', 'junior', '26-30', '46K-50K', 3],
    ['Systems', 'senior', '41-45', '66K-70K', 3],
    ['Marketing', 'senior', '36-40', '46K-50K', 10],
    ['Marketing', 'junior', '31-35', '41K-45K', 4],
    ['Secretary', 'senior', '46-50', '36K-40K', 4],
    ['Secretary', 'junior', '26-30', '26K-30K', 6],
]

# Defining mappings for ordinal features
age_mapping = {'21-25': 0, '26-30': 1, '31-35': 2, '36-40': 3, '41-45': 4, '46-50': 5}
salary_mapping = {'26K-30K': 0, '31K-35K': 1, '36K-40K': 2, '41K-45K': 3, '46K-50K': 4, '66K-70K': 5}

# Preparing data for encoding
departments = [row[0] for row in data]
ages = [age_mapping[row[2]] for row in data]
salaries = [salary_mapping[row[3]] for row in data]
statuses = [1 if row[1] == 'senior' else 0 for row in data] # Encode status as 1 for 'senior' and 0 for 'junior'

# One-hot encode the 'Department' column
encoder = OneHotEncoder(sparse_output=False)
department_encoded = encoder.fit_transform(np.array(departments).reshape(-1, 1))

# Combining features to create the final training data
X_train = np.hstack((department_encoded, np.array(ages).reshape(-1, 1),
np.array(salaries).reshape(-1, 1)))
Y_train = np.array(statuses)

# Defining and training the MLP model
myANN = MLPClassifier(
    hidden_layer_sizes=(4,), # 1 hidden layer with 4 nodes
    activation='relu', # ReLU activation function for the hidden layer
    solver='adam', # Adam optimizer
    learning_rate_init=0.01, # Initial learning rate
    max_iter=500, # Maximum number of iterations
    random_state=42 # For reproducibility
)

# Training the model
myANN.fit(X_train, Y_train)
```

```

# Predicting on training data
y_pred = myANN.predict(X_train)

# Displaying predictions and training data accuracy
print("Predictions on training data:", y_pred)
print("Training accuracy:", myANN.score(X_train, Y_train))

```

Output:

```

Predictions on training data: [1 0 0 0 1 0 1 1 0 1 0]
Training accuracy: 1.0

```

If we use encoding arrays as X_train and Y_train, then:

```

from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import OneHotEncoder, StandardScaler
import numpy as np

X_train = np.array([ [1, 0, 0, 0, 2, 4], [1, 0, 0, 0, 1, 0], [1, 0, 0, 0, 2, 1], ])
# must add all other 165 data points based on encoding
Y_train = np.array([1, 0, 0, 1, 0]) # Example testing data - binary values for class label

myANN = MLPClassifier(hidden_layer_sizes=(4,), # Single hidden layer with 4 nodes
                      activation='relu',      # ReLU activation function
                      solver='adam',          # Adam optimizer
                      learning_rate_init=0.01, # Learning rate
                      max_iter=500,           # Increase if needed for convergence
                      random_state=42)

myANN.fit(X_train, Y_train)

Y_pred = myANN.predict(X_train)

print("Predicted values:", Y_pred)

```

2) Support Vector Machine Linear Classifier:

```

import pandas as pd
from sklearn.svm import SVC
from sklearn.preprocessing import OneHotEncoder, LabelEncoder
import numpy as np

# This is the dataset
data = {
    "department": ["Sales", "Sales", "Sales", "Systems", "Systems", "Systems", "Systems", "Marketing",
                  "Marketing", "Secretary", "Secretary"],
    "status": ["senior", "junior", "junior", "junior", "senior", "junior", "senior", "senior", "junior",
              "senior", "junior"],
    "age": ["31-35", "26-30", "31-35", "21-25", "31-35", "26-30", "41-45", "36-40", "31-35", "46-50",
           "26-30"],
    "salary": ["46K-50K", "26K-30K", "31K-35K", "46K-50K", "66K-70K", "46K-50K", "66K-70K",

```

```

"46K-50K", "41K-45K", "36K-40K", "26K-30K"],
    "count": [30, 40, 40, 20, 5, 3, 3, 10, 4, 4, 6]
}

df = pd.DataFrame(data)
expanded_df = df.loc[df.index.repeat(df['count'])].reset_index(drop=True)

age_map = {"21-25": 0, "26-30": 1, "31-35": 2, "36-40": 3, "41-45": 4, "46-50": 5}
salary_map = {"26K-30K": 0, "31K-35K": 1, "36K-40K": 2, "41K-45K": 3, "46K-50K": 4, "66K-70K": 5}
expanded_df['age'] = expanded_df['age'].map(age_map)
expanded_df['salary'] = expanded_df['salary'].map(salary_map)

ohe = OneHotEncoder(sparse_output=False, drop='first')
department_encoded = ohe.fit_transform(expanded_df[['department']])

le = LabelEncoder()
status_encoded = le.fit_transform(expanded_df['status'])

X = np.hstack([department_encoded, expanded_df[['age', 'salary']].values])
y = status_encoded

# Defining and training the SVM classifier
mySVM = SVC(kernel='linear', C=1.0)
mySVM.fit(X, y)

# Obtaining support vectors and labels
support_vectors = mySVM.support_vectors_
support_vector_indices = mySVM.support_
support_vector_labels = y[support_vector_indices]

# Calculating weights and bias
weights_vector = mySVM.coef_[0]
bias_value = mySVM.intercept_[0]

# Print the results in the specified format
print("Support Vectors:")
print("There are", len(support_vectors), "support vectors")
print("\nAttributes and Labels:")
print("Attribute 1 | Attribute 2 | Attribute 3 | Attribute 4 | Attribute 5 | Label")
print("-" * 78)

# Print support vectors and labels
for sv, label in zip(support_vectors, support_vector_labels):
    print(" ".join(f"{attr:.2f}" for attr in sv), f"| {label}")

# Number of attributes displayed
total_attributes = len(weights_vector) # This should give the number of features
print("\nWeights (Beta values) and Bias (Intercept):")
print("Attribute 1 | " + " | ".join([f"Attribute {i+2}" for i in range(total_attributes-1)]))
print("-" * 78)
print(" ".join(f"{weight:.2f}" for weight in weights_vector))

print("\nBias (Intercept):")
print(f"{bias_value:.2f}")

```

Output:

Support Vectors:
There are 6 support vectors

Attributes and Labels:

Attribute 1	Attribute 2	Attribute 3	Attribute 4	Attribute 5	Label
0.00	0.00	1.00	1.00	4.00	0
0.00	0.00	0.00	2.00	3.00	0
0.00	0.00	0.00	2.00	3.00	0
1.00	0.00	0.00	2.00	4.00	1
0.00	0.00	1.00	2.00	5.00	1
0.00	1.00	0.00	5.00	2.00	1

Weights (Beta values) and Bias (Intercept):

Attribute 1	Attribute 2	Attribute 3	Attribute 4	Attribute 5
0.96	0.17	-0.09	0.96	1.04

Bias (Intercept):
-6.04

Final Answer:

There are 6 support vectors:

Support Vectors	Attribute 1	Attribute 2	Attribute 3	Attribute 4	Attribute 5	Label
1	0.00	0.00	1.00	1.00	4.00	0
2	0.00	0.00	0.00	2.00	3.00	0
3	0.00	0.00	0.00	2.00	3.00	0
4	1.00	0.00	0.00	2.00	4.00	1
5	0.00	0.00	1.00	2.00	5.00	1
6	0.00	1.00	0.00	5.00	2.00	1

	Attribute 1	Attribute 2	Attribute 3	Attribute 4	Attribute 5
Weights	0.96	0.17	-0.09	0.96	1.04
Bias	-6.04				